# Test Specification

# NestUp

What is tested, why, and what is not

**Internet Technologies — Become a Full-Stack Engineer**

RUNI CS 2026 · Final project

Live product **nestup-kappa.vercel.app**

Repository **github.com/nestupweb/nestup**

**Suite:** 97 test files · 740 tests · Vitest + React Testing Library (jsdom) + Playwright (real browser) **Run:** `npm test` — see §8 for the reason that `npx vitest` must never be used directly.

---

## 1. What "working" means for this product

Before listing the tests themselves, it is worth stating explicitly what the suite is actually defending, because this determines what is tested and what is not tested.

NestUp can fail in three ways which matter, presented here in descending order of cost:

1. **A member sees the data of another member.** This failure is unrecoverable, because the entire trust proposition of the product collapses.
2. **A core flow silently performs the wrong operation.** For example, a message which does not arrive, a swiped room which returns to the deck, or an edit to a listing which also erases the chats of the owner.
3. **A screen is incorrect or unattractive.** This is embarrassing, but it is inexpensive to correct.

The suite is weighted accordingly. Access control and business rules are tested rigorously, whereas presentation is tested only where it encodes an actual rule, such as a button which must be disabled, rather than for the sake of its appearance.

**The following is deliberately not tested:** every line of every component, the exact wording of the copy, the exact colors (except in cases where a rule depends on them, as in `map-basemap.test.ts`), and third-party libraries. The assignment requires that the main flows be tested and not that every line be tested. Moreover, a suite which asserts on wording eventually becomes a suite which nobody is willing to modify.

---

## 2. Feature tests — the main flows

### Authentication and onboarding

| TEST FILE | WHAT IT DEFENDS |
| --- | --- |
| `auth-form.test.tsx` | The sign-up and sign-in forms, including field state, submission and error display |
| `signup-confirmation.test.ts` | Registration must **not** return a usable session. The row is created, a code is mailed, and access waits for confirmation |
| `auth-password-actions.test.ts` | The forgot-password and reset flows |
| `auth-confirm-route.test.ts` | Every shape of emailed link arrives correctly: signup leads to onboarding, recovery leads to reset, and an invalid link leads to `/login` with the appropriate notice |
| `auth-email-templates.test.ts`, `auth-mail.test.ts`, `mail.test.ts` | The mail templates and the sender behave as intended |
| `password-input.test.tsx` | The show and hide toggle |
| `redirect.test.ts` | The function `sanitizeNextPath`, which provides open-redirect hardening |

## Profile

| TEST FILE | WHAT IT DEFENDS |
| --- | --- |
| `profile-action.test.ts` | Saving a profile writes the correct rows and invalidates the correct caches |
| `profile-required-fields.test.tsx` | Which fields prevent a save, together with the banner which explains this |
| `profile-form-sticky.test.tsx` | A rejected form retains whatever the member typed. React 19 resets forms by default, and this test is the guard against that behavior |
| `about.test.tsx`, `contact-row.test.tsx` | Private details and contact visibility |
| `daily-life.test.tsx`, `daily-life-reminder.test.tsx` | The lifestyle questionnaire and the reminder to complete it |
| `profile-amenities.test.ts`, `profile-safe-room`, `interests-picker.test.tsx` | The preference inputs |
| `profile-avatar.test.tsx`, `photo-picker.test.tsx`, `photo-order.test.tsx` | Photo upload, ordering and the lightbox |
| `profile-tabs-history.test.tsx`, `my-listing.test.tsx` | The profile tabs |
| `people.test.ts` | The public profile of another member |

## Listings

| TEST FILE | WHAT IT DEFENDS |
|---|---|
| `listing-query.test.ts` | The browse query, including filters, sorting and pagination |
| `listing-actions.test.tsx`, `listing-status-action.test.ts` | Publishing, pausing and resuming |
| `delete-listing-action.test.ts` | Soft removal |
| `mark-taken.test.tsx`, `listing-taken.test.ts` | The "taken" state |
| `listing-title.test.ts`, `property-tile.test.tsx`, `listing-gallery.test.tsx` | The presentation of a room |
| `filter-bar.test.tsx`, `sort-menu.test.tsx` | The filter and sort controls |
| `save-button.test.tsx` | Hearts, both in the signed-in state and in the signed-out state |
| `roommate-tag-picker.test.tsx`, `roommate-count.test.ts`, `roommates-fit-rooms.test.ts` | The rules governing household composition |
| `co-posters.test.ts`, `co-posters-action.test.ts`, `co-poster-invites.test.tsx` | Co-ownership and invitations |

## Matching

| TEST FILE | WHAT IT DEFENDS |
|---|---|
| `compatibility.test.ts` | The scoring function, including each weighted component and the convention that a missing preference receives approximately 60% |
| `swipe-rank.test.ts` | Deck ordering and the `MIN_DECK_SCORE` threshold |
| `swipe-deck.test.tsx` | The deck interface, the like and skip operations, and the empty state |
| `affinity.test.ts` | Interest overlap |
| `apartment-prefs.test.ts`, `no-city-prompt.test.tsx` | The requirement of a preferred city, together with its prompt |
| `seed-data.test.ts` | **Measures the real deck across all 124 cities**, so that a town cannot silently become unmatchable |

## Chat and viewings

| TEST FILE | WHAT IT DEFENDS |
|---|---|
| `send-message-action.test.ts` | Sending a message, and the idempotent retry of an identical `client_id` |
| `chat-realtime.test.tsx` | The token reaches the socket *before* the channel joins, and there is exactly one invalidation per burst |
| `chat-visible.test.ts` | The per-member "delete chat" cutoff |
| `chat-outbox.test.ts`, `message-composer.test.tsx`, `chat-media.test.ts`, `chat-format.test.ts` | Optimistic sending, attachments and formatting |
| `message-owner.test.tsx` | The "message the household" entry point |
| `viewing-card.test.tsx`, `viewing-details.test.tsx`, `availability.test.ts`, `calendar.test.ts` | Proposing, approving, availability windows and calendar export |

## Settings and moderation

| TEST FILE | WHAT IT DEFENDS |
|---|---|
| `account-actions.test.ts`, `account-section.test.tsx` | Changing the email address and the password |
| `danger-zone.test.tsx` | Account deletion, **including the listing-handover picker in all three of its shapes**, namely no heir, exactly one heir, and several heirs |
| `setting-toggle.test.tsx`, `settings-gear.test.tsx` | The settings controls |
| `moderation.test.ts` | Reporting, blocking, unblocking and suspension |
| `notify.test.ts` | The new-match notification |

## 3. Invalid-input tests

| AREA | WHAT IS ASSERTED |
|---|---|
| `validation.test.ts` | Every Zod schema, including required fields, lengths, ranges, enums and coercion |
| `profile-required-fields.test.tsx` | A save which is missing a required field is refused together with a field-level message |
| `send-message-action.test.ts` | An empty message, an over-length message, a malformed conversation identifier, and a photo path located outside the folder of the conversation itself |
| `listing-query.test.ts` | Meaningless query strings fall back to the default values instead of producing an error, because `.catch()` is applied to every filter |
| `amount-input.test.tsx`, `phone-field.test.tsx`, `date-picker.test.tsx`, `city-combobox.test.tsx` | The rejection of non-numeric rent, malformed telephone numbers, impossible dates and unknown cities |
| `photo-rules.test.ts`, `photo-check.test.ts` | An incorrect file type, an oversized file, and a photograph whose **content** does not correspond to its declared tag |
| `redirect.test.ts` | Targets such as `?next=https://evil.example`, and other off-site destinations, are rejected |
| `image-client.test.ts` | The boundaries of client-side compression and of HEIC conversion |

## 4. Business-process tests

These tests assert the rules which make the product itself correct, and not merely the code.

| RULE | WHERE IT IS ASSERTED |
|---|---|
| A swiped room never returns to the deck | `swipe-rank.test.ts`, `cache-invalidation.test.ts` |
| Only rooms scoring 60 or above enter the deck | `swipe-rank.test.ts` |
| A seeker who has no preferred city receives no deck | `apartment-prefs.test.ts` |
| One person may have one active listing | `roommates-fit-rooms.test.ts`, `co-posters.test.ts` |
| The number of roommates cannot exceed the number of rooms | `roommates-fit-rooms.test.ts` |
| Deleting an account **transfers** a shared listing rather than destroying it | `danger-zone.test.tsx` |
| A member who already has a live listing cannot inherit a second one | `danger-zone.test.tsx` |
| Deleting a chat hides it for one side only, and the next message restores it | `chat-visible.test.ts` |
| A blocked member disappears from the decks and from the search in both directions | `moderation.test.ts` |
| A retried message carrying the same `client_id` is not posted twice | `send-message-action.test.ts` |
| Every city is able to fill a deck | `seed-data.test.ts` |

### Cache invalidation treated as a business rule

The file `cache-invalidation.test.ts` deserves a separate explanation. It asserts the **scope of impact** of every mutation: that adding a room to the liked list touches exactly `saved:<me>` and `profile:<me>` and nothing further; that no listing mutation reaches into the chat data; and that no chat mutation reaches into the deck or into the room list.

This test exists because of a genuine defect which occurred earlier. Mutations used to respond to every write with a large number of `revalidatePath` calls, and consequently editing a room also discarded the chats and the profile of that member. The test encodes the rule which replaced that behavior.

---

## 5. Permission tests

This is the category with the highest value, since the most severe failure mode of the product is the exposure of data across members.

| TEST | WHAT IT DEFENDS |
|---|---|
| `cache-invalidation.test.ts`, specifically the case *"every per-member tag is scoped to the member who acted"* | Every `deck:`, `profile:`, `saved:` and `chat:` tag carries the identifier of the acting member. A tag which omitted that identifier would become **a single shared cache key for the entire application**, which is precisely the form that a cross-user leak takes |
| `cached-session-boundary.test.ts` | **No Server Action is permitted to authorize itself using the cached session.** Identity is cached for the purpose of *rendering* only, and every write still performs the uncached check. Both spellings compile successfully, and therefore only a test is able to enforce this boundary |
| `cached-session-boundary.test.ts` (continued) | The suspension check still reads the `suspensions` table, and both gates still redirect on the basis of it |
| `signout-clears-cache.test.ts` | Signing out answers with **303**, which forces a full document load, and this is the only mechanism which empties the private caches held by the browser. The test also asserts that there is **no GET handler**, so that a logout cannot be triggered from another site |
| `member-actions.test.tsx` | The Log out button posts to the route handler rather than to a Server Action, since a soft redirect would leave the caches alive |
| `moderation.test.ts` | A member cannot remove his or her own suspension, and blocking is mutual |
| `profile-action.test.ts` | Only the owner is able to edit a profile |
| `co-posters-action.test.ts` | Only a member who was invited is able to accept an invitation |

**The layer which these tests cannot reach** is RLS itself, which is enforced by Postgres, whereas the unit suite mocks the Supabase client. RLS is therefore verified by the real-browser checks described in §7, and by the fact that the policies constitute the *only* path to the data. This is discussed further in the Security document.

## 6. Database tests

| TEST | WHAT IT DEFENDS |
| --- | --- |
| `seed-data.test.ts` | A **sha256 fingerprint** computed over the first 92 seed records. Consequently, re-running `npm run seed` can never silently modify existing demo data |
| `seed-data.test.ts` (continued) | Re-measures deck coverage across all 124 cities using the real `buildDeck` function |
| `city-centres.test.ts` | Protects all 124 generated city coordinates against drift |
| `listing-query.test.ts` | The translation from filters into SQL, including the arithmetic of the pagination `range()` |
| `cache-lifetimes.test.ts` | Every `cacheLife` declaration specifies `stale ≥ 300s`, which is the App Shell threshold, and the router cache is not shorter than the data caches |
| `roommates-fit-rooms.test.ts`, `roommate-count.test.ts` | The denormalized household columns which are maintained by triggers |

# 7. Edge cases

| CASE | TEST |
| --- | --- |
| An empty deck, either because every room was already swiped or because no room reaches a score of 60 | `swipe-deck.test.tsx`, `swipe-rank.test.ts` |
| An empty inbox, and a chat which was deleted and then revived by a new message | `chat-visible.test.ts` |
| A room which is removed while a conversation about it is open | `listing-taken.test.ts`, together with the second `listings` SELECT policy |
| Concurrent creation of a conversation, producing a unique-constraint race | `findOrCreateConversation` falls back to re-reading the row |
| A duplicated message send | `send-message-action.test.ts` |
| Account deletion with zero, one or several eligible heirs | `danger-zone.test.tsx` |
| The realtime socket joining before the token arrives | `chat-realtime.test.tsx` |
| A failing realtime synchronization must not throw inside an effect | `chat-realtime.test.tsx` |
| A stale `?needs=cities` parameter remaining in the URL after the member has already corrected it | `no-city-prompt.test.tsx` |
| A listing which has no photographs | `listing-gallery.test.tsx` |
| Navigation must remain within a single tab | `same-tab-navigation.test.ts` together with `npm run check:nav` |

### Real-browser checks (Playwright)

Certain properties cannot be asserted inside jsdom, because they require a real engine, a real GPU canvas, or the real deployment.

| SCRIPT | WHAT IT PROVES |
|---|---|
| `npm run check:map` | Both themes render correctly, and the script counts red pixels **directly from the WebGL canvas**, since a GL layer has no DOM which could be queried |
| `npm run check:nav` | No internal link opens a new tab |
| `.superpowers/e2e/nav-cache.mjs` | **21 assertions executed against the live site:** signing out destroys the JavaScript context; `/swipe` redirects after a logout; the back button cannot restore a signed-in page; and a second member using the same tab sees only his or her own data |
| `.superpowers/e2e/chat-freshness.mjs` | A real message reaches the thread *and* updates the cached inbox row, and it survives navigating away and returning |
| `.superpowers/e2e/skeleton-duration.mjs` | Measures how long a loading skeleton is actually visible on screen, per tab |

## 8. How to run the suite

```
npm test                              # 97 files, 740 tests
npm test -- --testTimeout=25000       # use this if userEvent tests time out (see below)
npx tsc --noEmit                      # types
npx eslint                            # lint
```

There are two environment traps, both of which were discovered through experience, and both of which are worth knowing before accepting a red result as genuine:

1. **Always use `npm test`, and never `npx vitest run`.** The npm script sets `NODE_OPTIONS=--no-experimental-webstorage`. Without this flag, Node 25 installs its own inert `localStorage` global which jsdom does not replace, and consequently `theme-toggle.test.tsx` fails with `localStorage.clear is not a function`, which is a failure that says nothing at all about the component itself.
2. **OneDrive synchronization starves `userEvent`.** The project resides in a synchronized folder, and when OneDrive is busy, approximately half a dozen interaction tests exceed the default limit of 5 seconds. The suite should therefore be re-run with `--testTimeout=25000` before a timeout is treated as a real failure.

## 9. Known gaps

These are stated plainly, because a test document which claims complete coverage is not credible.

- **The RLS policies are not exercised by the unit suite.** The suite mocks the Supabase client, and therefore the policies themselves are covered only by the real-browser checks and by manual review. A future improvement would be a test which executes real queries as two different members against a dedicated test project.
- **There is no automated end-to-end journey.** Playwright is used for targeted checks rather than for a complete script covering registration, profile creation, swiping, chatting and scheduling a viewing.
- **Email delivery is mocked.** The templates and the sender are tested, whereas actual arrival in an inbox is verified manually.
- **The Gemini photo check is mocked.** Its contract is tested, but the judgment of the model itself is not.
- **Load has not been measured.** The reasoning about scale is analytical; see the Scale document.

---